

VisualNeedle: Benchmarking Active Visual Search in Information-Dense Scenes

Jingru Chen^{1,2*} Yiming Liu^{1*} Mingtao Chen¹ Sijie Chen^{1,3}
 Richeng Xuan¹ Liang Yang¹ Zhichao Hu¹ Fanyang Lu^{1†}
¹Hunyuan, Tencent ²Peking University ³Zhejiang University

Abstract

Frontier multimodal large language models (MLLMs) have been reported to achieve over 90% accuracy on fine-grained perception benchmarks. However, such scores do not necessarily imply faithful use of visual evidence. Prior studies have identified three shortcuts that inflate benchmark performance. First, linguistic priors and lexical cues in questions often enable models to infer plausible answers without seeing the image. Second, coarse global semantics from the visual encoder can bypass fine-grained local details. Third, in some “think-with-images” benchmarks, corrupting the intermediate images returned by visual tools barely affects the final answer. These findings suggest that higher input resolution or larger question pools alone do not elicit genuine active visual search. To address this, we introduce VisualNeedle, a challenging, information-dense, and fine-grained benchmark for scenes where critical evidence is spatially constrained to minute regions and not discernible at a glance. We further propose a counterfactual crop-black setting, which replaces crops returned by tools with black images of the same size, to test whether tool-enabled performance truly relies on intermediate visual evidence. We evaluate 9 prominent MLLMs across three settings: no-tool, standard tool-enabled, and crop-black. No-tool accuracy stays below 20%, and the best tool-enabled model reaches only 56.01%, still trailing the 63.00% human majority-vote accuracy. These results reveal persistent limitations in fine-grained visual search, while the crop-black ablation confirms that success on VisualNeedle hinges on genuine intermediate visual evidence.

1 Introduction

Multimodal large language models (MLLMs) have made rapid progress on visual understanding tasks, and the research community is increasingly moving from passive image understanding, or “thinking about images” [Alayrac et al., 2022, Li et al., 2023, Liu et al., 2023b], toward a more active visual reasoning paradigm, or “thinking with images” [Su et al., 2025, Zheng et al., 2025, Zhang et al., 2025, Guo et al., 2025, Li et al., 2025a, Hong et al., 2025]. Under this paradigm, a model no longer receives the entire image once and directly produces an answer; instead, it is expected to actively localize relevant regions, acquire local visual evidence through operations such as cropping, zooming, and code execution, and incorporate these intermediate visual observations into subsequent reasoning. Correspondingly, recent fine-grained perception and high-resolution visual benchmarks have begun to design tasks around this active visual capability: they ask models to identify small objects, read distant or tiny text, judge local attributes, and understand subtle spatial relations in complex images, thereby testing whether models can move from a global scene to the local evidence needed for answering [Yu et al., 2026, Kanade and Ganu, 2025, Grover et al., 2026, Feng et al., 2026,

*Equal contribution.

†Correspondence to dylannlu@tencent.com.

All data and code will be released soon.

Li et al., 2025b]. Leading systems now achieve over 90% accuracy on several established benchmarks, suggesting at first glance that fine-grained visual perception, and even active visual search, may be approaching saturation. This raises a more fundamental evaluation question: do these high scores indicate that models can actively search for, localize, and use fine-grained visual evidence?

This concern is not merely speculative. Three recent findings suggest that benchmark accuracy can be decoupled from genuine visual evidence use. First, language priors and lexical cues in the question can lead models to produce plausible answers without seeing the image [Asadi et al., 2026]. Second, visual encoders may retain coarse global semantics while missing fine local details, allowing models to rely on scene-level gist instead of resolving the decisive clue [Tong et al., 2024]. Third, in “think-with-images” pipelines, corrupting intermediate visual outputs can leave final decisions largely unchanged, indicating that tool use does not necessarily imply reliance on the returned visual evidence [Liu et al., 2025]. Together, these failure modes make high accuracy an ambiguous signal: a benchmark may appear saturated while still failing to test whether models actively search for, localize, and use fine-grained visual evidence.

This makes active visual search a measurement problem, not merely a task difficulty problem. Simply increasing image resolution or adding more questions does not guarantee that models must engage with local evidence. A benchmark for this setting should instead be constructed so that shortcut solutions fail by design: the question alone should be insufficient, a one-shot global view should not reveal the answer, and tool-enabled success should depend on the visual content returned during interaction.

Guided by this principle, we introduce **VisualNeedle**, a 300-question benchmark for active visual search in information-dense scenes, including urban street views, dense documents, shelves, book-cases, and maps. Each question satisfies an answer-not-visible-at-a-glance criterion and is anchored to a small but decisive local visual clue. The benchmark spans five complementary abilities: OCR, color recognition, entity recognition, spatial relation judgment, and occluded object recognition, all requiring models to move from a cluttered global scene to specific local evidence before answering. This design makes each instance depend on finding the right local evidence rather than recognizing the overall scene type or exploiting answer regularities.

Beyond static evaluation, VisualNeedle directly tests whether models use the evidence discovered during search. We formulate the task as a multi-step tool-use process and introduce the crop-black setting, which replaces returned crop images with all-black images of the same size while preserving the interaction trajectory. Comparing this setting with standard Crop-based evaluation reveals whether tool-enabled gains come from visual evidence rather than from the act of calling tools itself. Because the ablation preserves the tool trajectory and returned image dimensions, performance changes isolate the contribution of the returned visual content. Figure 1 illustrates the overall task format, search process, and benchmark challenges.

We evaluate 9 mainstream MLLMs under four settings: text-only tests question-level priors; no-tool tests one-shot full-image perception without tools; tool-enabled provides real tool-based local evidence; and the crop-black setting preserves the tool-use protocol while replacing Crop outputs with all-black patches. The results validate the benchmark design. Text-only accuracy stays below 10%, showing that question-level priors are insufficient; no-tool accuracy stays below 20%, showing that one-shot global perception is also insufficient. With tools, the strongest model, Gemini 3.1 Pro, reaches 56.01%, still below the human majority accuracy of 63.00%. The crop-black setting further shows that successful performance on VisualNeedle depends on real intermediate visual evidence. Together, these results test the three intended failure modes: language priors, one-shot global perception, and tool-output invariance. Additional trajectory and tool-use analyses reveal that models differ not only in whether they call tools, but also in whether they can localize evidence and integrate it reliably.

These findings demonstrate that the active visual search capability of current MLLMs in information-dense scenes remains fundamentally limited. Our contributions are threefold:

1. **We identify shortcut-driven mismeasurement in current fine-grained visual benchmarks and translate it into benchmark design constraints.** VisualNeedle is constructed so that language priors, coarse global semantics, and tool-output invariance are insufficient for reliable success.

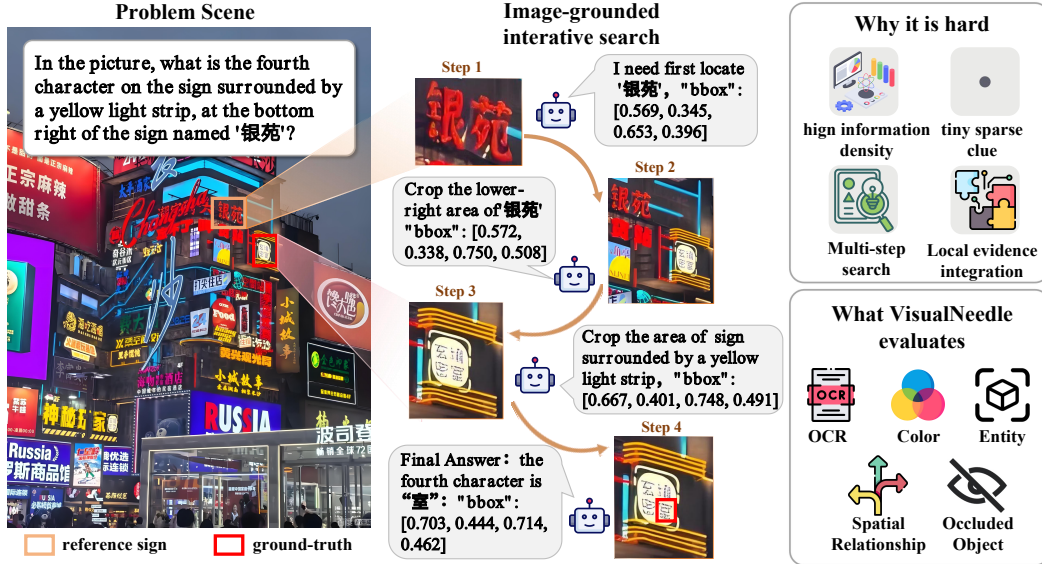


Figure 1: Overview of VisualNeedle and the interactive search workflow. Left: an example query in an information-dense scene. Middle: the step-by-step search process through sequential crops. Right: the core challenges and evaluated abilities.

2. **We build VisualNeedle, a 300-question benchmark for active visual search in information-dense scenes.** The benchmark covers five fine-grained abilities and requires models to locate small local evidence before answering.
3. **We introduce the crop-black setting to test intermediate visual evidence dependence.** The results show that success on VisualNeedle depends on the actual returned crops, exposing limitations in current MLLMs’ active visual search behavior.

2 Related Work

Our work builds upon and extends research in visual-language benchmarks, high-resolution perception, and visual tool use. We organize related work along three dimensions that highlight the unique contributions of VisualNeedle.

2.1 Visual-Language Benchmarks and Their Limitations

Early vision-language benchmarks such as MME [Fu et al., 2023], MMBench [Liu et al., 2023a], and MMMU [Yue et al., 2023] focused primarily on general visual question answering, where models typically receive the whole image once and produce a final answer. Recent benchmarks including MME-RealWorld [Zhang et al., 2024] and RealX-Bench [Hong et al., 2025] move toward harder real-world visual understanding by increasing image realism, resolution, and scene complexity. These benchmarks have been important for tracking the broad progress of MLLMs, but their evaluation remains largely final-answer based. As a result, high accuracy alone may not reveal whether a model grounds its answer in the necessary visual evidence.

2.2 High-Resolution and Fine-Grained Perception Benchmarks

Recent benchmarks have examined high-resolution and fine-grained perception from complementary perspectives. HR-Bench [Wang et al., 2024] studies information loss and perceptual hallucination caused by image resizing, while V* Bench [Wu and Xie, 2024] introduces active visual search by requiring models to locate target objects. VisualOverload [Gavrikov et al., 2025] further stresses MLLMs with extremely dense visual scenes. More recent benchmarks broaden this direction to fine-

grained reasoning over transit maps [Feng et al., 2026], multidimensional visual perception [Kanade and Ganu, 2025], fine-grained pattern recognition [Grover et al., 2026], general fine-grained image tasks [Yu et al., 2026], and multi-hop reasoning over ultra-high-resolution images [Li et al., 2025b].

Together, these works show that resolution, visual density, local detail, and spatial grounding are central to evaluating fine-grained perception. However, they do not simultaneously require shortcut-resistant question design, diverse information-dense real-world scenes, and a diagnostic test of whether tool-enabled success depends on returned intermediate visual content. VisualNeedle is designed to combine these requirements in a single benchmark.

2.3 Visual Tool Use and Interactive Evaluation

The growing capability of visual agents has spurred research on tool use and interactive evaluation. DeepEyes [Zheng et al., 2025] and Thyme [Zhang et al., 2025] use reinforcement learning to train models to call tools such as Crop, improving performance on fine-grained tasks. VisualToolBench [Guo et al., 2025] constructs open-ended tasks for general visual tool-use evaluation. TIR-Bench [Li et al., 2025a] extends evaluation to multiple task types involving tool use and image-processing reasoning. VTC-Bench [Zhu et al., 2026] provides OpenCV tools for evaluating composition and planning in multi-step visual tool chains.

These works establish tool use as an important capability for MLLMs, but successful tool calling does not by itself demonstrate reliance on the visual content returned by tools. VisualNeedle therefore focuses on a narrower but more diagnostic question: in dense local visual search, do models use the intermediate visual evidence obtained through interaction? The crop-black setting directly targets this question by preserving the tool-use trajectory while removing the returned visual content.

3 VisualNeedle Benchmark

VisualNeedle is designed as a constructive response to the three shortcuts identified above: language priors, coarse global semantics, and weak dependence on intermediate visual evidence. Rather than treating these shortcuts as post hoc diagnostic observations, we translate them into three benchmark requirements: the question alone should be insufficient, a one-shot global view should be insufficient, and tool-enabled success should depend on the visual content returned during interaction. The task definition, curation pipeline, and the crop-black setting are built around these requirements. The following subsections describe the resulting task protocol, data construction process, and dataset characteristics.

3.1 Task Definition

VisualNeedle formulates fine-grained visual perception as a sequential visual search problem rather than a one-shot VQA problem. Given a high-resolution image I and a question Q , the model starts from the full-image observation $o_0 = I$ and repeatedly chooses an action $a_t \in \{\mathbf{Crop}(bbox_t), \mathbf{Answer}(y_t)\}$: **Crop** returns a localized visual observation, while **Answer** terminates the interaction and produces the final answer. This protocol also enables the crop-black setting, where cropped observations are replaced with all-black patches of the same size while the interaction trajectory is preserved. This tests whether the model’s answer depends on intermediate visual evidence rather than merely on the act of calling tools; the complete stepwise protocol is specified in Appendix Algorithm 1.

3.2 Design Constraints and Curation Pipeline

VisualNeedle is constructed through a constraint-driven curation protocol rather than standard image-question sampling. Each sample is required to satisfy three anti-shortcut constraints that directly instantiate the requirements above. First, *question-alone insufficiency*: the answer should not be recoverable from the wording of the question or its lexical cues. Second, *one-shot global-view insufficiency*: the decisive evidence should not be reliably captured from the downsampled whole-image view. Third, *returned-evidence dependence*: the local evidence discovered during interaction should be visually inspectable, so that tool-enabled performance can be tested against the crop-black setting. These constraints guide the following four-stage pipeline.

1. **Source image selection.** We first collect information-dense images from diverse real-world scenes, including complex street views, dense document scans, shelves packed with products, bookcases, and panoramic maps. Image selection is not driven by resolution alone. Instead, we prioritize scenes with rich local structure, many candidate entities, diverse object categories, and dense segmentation regions, so that coarse global semantics are insufficient for reliable answering.
2. **Needle mining.** During data construction, dataset curators inspect full-resolution images to identify small but recognizable visual evidence, which we refer to as “needles.” A needle may be distant text on a storefront sign, the title of a book on a shelf, a local attribute of a person, or the category of a partially occluded object. This step anchors each question to concrete local evidence and turns the task from passive image understanding into active visual search.
3. **Question construction.** Around each mined needle, the data-construction team writes questions covering five core ability categories: text recognition, color recognition, entity recognition, spatial relation judgment, and occluded object recognition. The question is written so that lexical cues alone are insufficient to infer the answer, while the correct answer becomes clear once the target region is localized and inspected. Many questions require progressive search and multiple perceptual operations, such as first locating a person in a crowded scene and then judging the color of that person’s clothing.
4. **Annotation and verification.** Each candidate sample undergoes multi-round cross-checking by different members of the data-construction and verification team. The verification process checks that the wording is unambiguous, the answer is unique, the target evidence is visible under appropriate local observation, and the answer is not visible at a glance from the original image scale. This construction-stage verification is separate from the human-baseline annotation used in our evaluation: the 12 external human-baseline annotators did not participate in image collection, question authoring, bounding-box drawing, or quality filtering, and they answered the final evaluation items independently without access to ground-truth answers or construction-time metadata.

3.3 Dataset Composition and Diagnostic Coverage

VisualNeedle contains 300 curated questions from information-dense scenes, including street views, documents, shelves, bookcases, and maps. Its examples span five fine-grained ability categories: text recognition, color recognition, entity recognition, spatial relation judgment, and occluded object recognition. Rather than serving as broad topical coverage alone, these categories are selected to expose distinct failure modes in active visual search.

Text recognition probes tiny text localization and reading; color recognition tests localized attribute grounding; entity recognition measures target localization in cluttered scenes; spatial relation judgment requires local relational reasoning after search; and occluded object recognition evaluates inference from incomplete local evidence. Together, these categories make each example a diagnostic probe of whether the model can move from a global scene to the small visual clue needed for the answer.

Table 1 further clarifies where the difficulty of VisualNeedle comes from. The benchmark is not designed to be difficult merely because it has more questions or the highest raw resolution. Instead, its difficulty comes from dense local visual structure: many candidate entities, diverse object categories, fine segmentation regions, and high visual complexity within each image. Although VisualNeedle is not the highest-resolution benchmark, it has substantially denser local structure, as reflected by DINO-X entity and category counts, SAM segmentation regions, and the Complexity in Complexity metric [Sarıtaş et al., 2025]. This supports the central goal of evaluating active visual search in scenes where coarse global perception is insufficient.

4 Experiments

4.1 Experimental Setup

We evaluate 9 mainstream models on all 300 VisualNeedle questions, covering closed-source commercial models such as Gemini 3.1 Pro and GPT-5.4 as well as open-source models such as Qwen3-VL-



Figure 2: Examples from VisualNeedle showing how questions require moving from a information-dense global scene to a small piece of local visual evidence. The examples probe complementary failure modes, including OCR, localized color grounding, spatial relation reasoning, and visible or occluded object recognition.

Table 1: Comparison between VisualNeedle and existing related benchmarks in resolution and complexity metrics.

Benchmark	#Questions	Resolution (MP)	DINO-X Entities	DINO-X Categories	SAM Segments	Complexity in Complexity
V* Bench	191	4.05	79.86	17.76	169.46	0.49
HR-Bench (8K)	1600	39.44	53.11	10.40	140.62	0.40
TIR-Bench	1215	3.61	35.26	8.76	186.73	0.37
VisualToolBench	1204	3.55	42.94	6.38	159.16	0.39
VisualOverload	2720	7.78	129.48	23.29	187.02	0.52
MME-RealWorld	13k	11.47	86.67	13.92	148.80	0.42
VisualProbe-Hard	106	20.88	110.03	18.41	224.58	0.53
VTC-Bench	680	1.36	26.31	4.71	102.08	0.27
VisualNeedle (ours)	300	7.46	122.79	24.55	352.17	0.73

235B. Following the benchmark design constraints, we consider four evaluation protocols: *text-only*, where the model receives only the question; *no-tool*, where the model receives a one-shot global view of the image and must answer directly; *tool-enabled*, where the model can call the **Crop** tool to obtain high-resolution local observations; and the *crop-black* setting, where the interaction protocol is preserved but returned crops are replaced with all-black images of the same size. All models are evaluated with the same evaluation harness to ensure consistency in multi-step interaction and answer judgment. We repeat the main no-tool and tool-enabled experiments three times under the same setting and report the mean values in Table 2; the corresponding run-to-run variance and 95% confidence intervals are reported in Appendix E. Results for the text-only diagnostic and the crop-black setting are also averaged over three runs and reported in Table 3.

4.2 Main Results

Active visual search remains challenging for current MLLMs. The main results support three observations. First, one-shot global perception fails on VisualNeedle. In the no-tool setting, all models remain below 20% accuracy, with the best model reaching only 17.89%. This confirms that the decisive evidence in VisualNeedle is usually not recoverable from a downsampled whole-image view, and that answering from global scene semantics is insufficient.

Table 2: Main experimental results on VisualNeedle, averaged over three repeated runs under the same setting. Δ denotes the absolute accuracy improvement after tools are introduced. Avg. Tool Calls denotes the average number of Crop calls per question, and Total Tool Calls denotes the total number of Crop calls over the 300-question evaluation split. The lower part of the table reports human-baseline results.

Model	w/ Tools Acc. (%)	w/o Tools Acc. (%)	Δ (%)	Avg. Tool Calls	Total Tool Calls
Gemini 3.1 Pro [Google, 2026]	56.01%	16.34%	39.67%	3.72	1117.33
Gemini 3 Flash [Google, 2025]	48.00%	15.12%	32.88%	3.90	1174.00
Doubao-Seed-2.0-Pro [ByteDance Seed Team, 2026]	36.80%	13.11%	23.69%	6.42	1923.00
Doubao-Seed-1.8 [ByteDance Seed Team, 2025]	31.78%	17.89%	13.89%	4.77	1435.67
GPT-5.4 [OpenAI, 2026]	31.23%	14.32%	16.88%	11.14	3339.33
GPT-5.2 [OpenAI, 2025]	28.31%	10.56%	17.75%	9.30	2795.67
Qwen3.5-Plus [Qwen Team, 2026a]	22.11%	15.00%	7.11%	3.51	1056.67
Kimi K2.5 [Moonshot AI, 2026]	19.32%	10.78%	8.55%	6.59	1974.33
Qwen3-VL-235B-A22B-Thinking [Qwen Team, 2026b]	11.44%	8.34%	3.10%	1.59	473.67
Human Baseline					
	Correct				
Metric	Count	Accuracy (%)		–	
Majority Vote@3	189/300	63.00%		–	
Pass@3	224/300	74.67%		–	

Second, visual tools help substantially but do not close the human gap. With Crop, stronger models obtain large gains: Gemini 3.1 Pro improves by 39.67 percentage points and reaches 56.01%, while Gemini 3 Flash reaches 48.00%. However, even the strongest model remains below the human majority accuracy of 63.00% and far below the human Pass@3 accuracy of 74.67%. This suggests that tools can expose missing local details, but current models still struggle to search for and integrate fine-grained evidence as reliably as humans.

Third, tool calling is not equivalent to effective evidence use. Some models call tools frequently but obtain limited gains: GPT-5.4 and GPT-5.2 average 11.14 and 9.30 Crop calls, respectively, yet remain far below the leading Gemini models. Conversely, Qwen3-VL-235B-A22B-Thinking calls tools infrequently and improves by only 3.10 points, suggesting premature stopping or ineffective localization. These results indicate that the core challenge is not merely whether a model can invoke Crop, but whether it can localize the relevant region and use the returned evidence reliably.

4.3 Intermediate Visual Evidence Dependence

This diagnostic directly tests the third design constraint: *returned-evidence dependence*. A tool-enabled model should improve because it uses the visual content returned by Crop, not simply because it follows a tool-use trajectory. We therefore compare standard tool-enabled evaluation with two diagnostic settings. In the *text-only* setting, the model receives only the question, which probes whether language priors and lexical cues are sufficient. In the *crop-black* setting, the model can still follow the same tool-use protocol, but every Crop call returns an all-black patch instead of the real cropped region. This removes intermediate visual evidence while preserving the interaction format.

Table 3 shows that VisualNeedle strongly depends on intermediate visual content. For all models, accuracy in the crop-black setting remains close to text-only accuracy, while real crops bring large gains for stronger models; for example, Gemini 3.1 Pro rises from 12.00% under the crop-black setting to 56.01% with real crops, and Gemini 3 Flash rises from 13.33% to 48.00%. These gaps show that tool gains on VisualNeedle come from using the returned visual evidence, not merely from following a tool-call trajectory.

The contrast with HR-Bench and V* is substantial. On V*, all models stay within 75.92%–87.43% even when crop content is replaced by black patches. HR-Bench is more mixed, but several strong models also retain high accuracy in the crop-black setting, such as Gemini 3 Flash at 84.00%, Doubao-Seed-2.0-Pro at 82.00%, and Doubao-Seed-1.8 at 86.50%. These results suggest that prior benchmarks can often still be solved from global context, question cues, or tool-use trajectories, whereas VisualNeedle makes the intermediate visual evidence necessary for tool-enabled success.

Table 3: Accuracy under text-only, the crop-black setting, and standard tool-enabled settings. text-only denotes accuracy when only the question is provided; the crop-black setting reports accuracy when Crop returns all-black patches; w/ tool denotes accuracy when Crop returns real cropped images. This diagnostic table reports accuracies averaged over three repeated runs under each setting.

Model	VisualNeedle (300)			HR-Bench			V*		
	text-only	crop-black	w/ tool	text-only	crop-black	w/ tool	text-only	crop-black	w/ tool
Gemini 3.1 Pro	7.67	12.00	56.01	47.00	71.00	88.00	46.07	84.29	97.38
Gemini 3 Flash	6.67	13.33	48.00	48.50	84.00	86.00	40.31	87.43	96.86
Doubao-Seed-2.0-Pro	8.00	13.33	36.80	52.00	82.00	93.00	31.94	86.91	93.72
Doubao-Seed-1.8	8.33	15.33	31.78	47.00	86.50	92.00	41.36	86.39	91.62
GPT-5.4	6.33	8.33	31.23	43.50	64.50	94.50	42.41	75.92	97.38
GPT-5.2	1.00	7.00	28.31	16.00	70.00	87.50	38.74	76.44	95.81
Qwen3.5-Plus	4.00	6.67	22.11	47.50	38.50	90.50	33.51	80.63	93.72
Kimi K2.5	6.67	10.00	19.32	42.50	36.50	86.00	36.13	82.72	93.19
Qwen3-VL-235B-A22B-Thinking	5.67	7.67	11.44	39.50	35.00	84.00	34.03	85.34	92.67

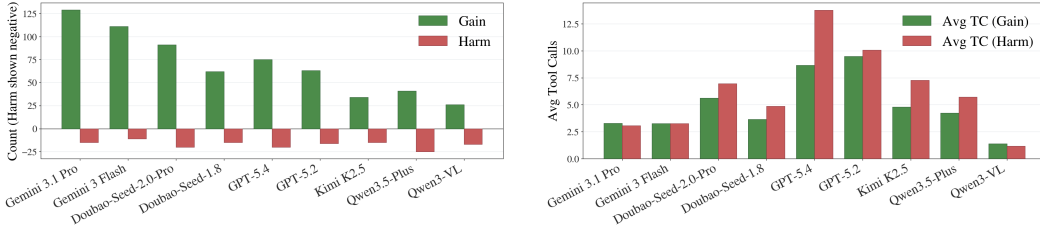


Figure 3: Stability and efficiency analysis of tool use. The left panel shows Gain/Harm analysis: Gain denotes questions answered incorrectly without tools but correctly with tools, and Harm denotes questions answered correctly without tools but incorrectly with tools. The right panel shows the average number of tool calls on Gain and Harm samples.

4.4 Tool Efficiency and Stability

Beyond final accuracy, we analyze tool-use behavior to understand why some models can call tools but still fail to answer correctly. This analysis is not a separate benchmark objective; instead, it provides behavioral evidence about search reliability and error amplification under the tool-enabled protocol. We adopt the Gain/Harm analysis framework [Ma et al., 2026], counting for each model the number of questions that become correct with tools compared with the no-tool setting (Gain) and the number of questions that become wrong (Harm). We also compute the average number of tool calls on Gain and Harm samples to characterize the interaction cost behind positive and negative changes.

Figure 3 shows that Gemini 3.1 Pro and Gemini 3 Flash have the most stable positive effects: their Gain counts are much higher than their Harm counts (129 vs. 15; 111 vs. 11), and their average numbers of calls on both Gain and Harm samples remain low. These models can therefore correct original errors with tools while usually completing effective search at low interaction cost.

By contrast, although GPT-5.4 and GPT-5.2 also benefit from tools to some extent, their gains come with substantially higher instability. They still have many Gain samples (75 and 63), but their Harm counts also rise to 20 and 16, and their average numbers of calls on Harm samples are higher. This suggests that their problem is not insufficient calling frequency; rather, their long interaction processes still struggle to converge reliably to the correct evidence. Weaker models further show that simply using tools is insufficient for improvement. For example, Qwen3-VL-235B-A22B-Thinking has only limited net benefit (26 Gain vs. 17 Harm), indicating that tool use itself can amplify errors when a model cannot use tools reliably.

4.5 Search Efficiency Analysis

We further characterize tool-use behavior through call intensity and call distribution. This analysis helps explain the gap between tool availability and effective evidence use: models may call Crop many times without converging to the right local evidence, or stop early before entering the relevant region. As shown in the left panel of Figure 4, Gemini 3.1 Pro and Gemini 3 Flash are closer to

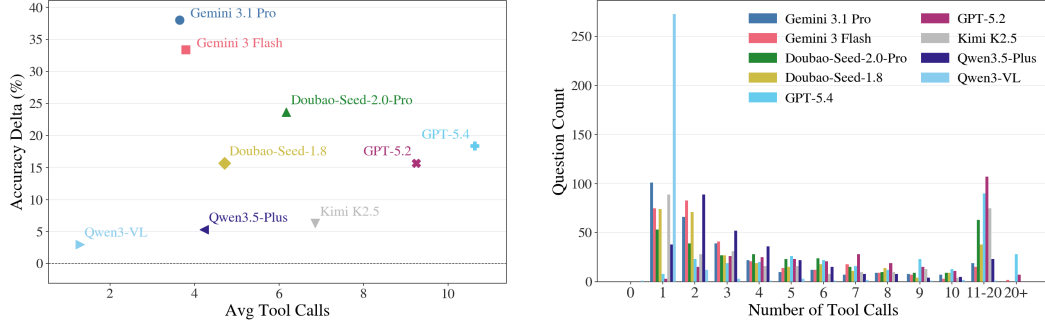


Figure 4: Analysis of models’ tool-use efficiency and calling behavior. The left panel shows the relationship between average tool calls and accuracy gain among samples with tool calls. The right panel shows the frequency distribution of questions across different tool-call intervals for each model, characterizing their tool-use behavior patterns.

the upper-left region, indicating that they obtain higher accuracy gains with fewer tool calls. This is consistent with their low average tool-call counts in the main table. By contrast, GPT-5.4 and GPT-5.2 have higher call intensity, but their gains do not increase correspondingly, showing that more frequent interaction does not automatically translate into more effective search. Meanwhile, Qwen3-VL-235B-Thinking has low average call counts, but its gain remains small, suggesting that low call frequency can also reflect premature stopping or ineffective localization.

The same difference is also visible in the call distribution shown in the right panel of Figure 4. Calls from Gemini 3.1 Pro and Gemini 3 Flash are mainly concentrated in the 1–3 range, suggesting that these models usually complete effective localization early. By contrast, GPT-5.4, GPT-5.2, and Moonshot-Kimi-2.5 have higher proportions in the 11–20 range, showing more pronounced long-tail calling behavior. These behavioral patterns explain why tool-enabled accuracy differs across models even under the same Crop interface: successful active visual search requires not only calling tools, but also stopping at the right local evidence and integrating it into the final answer.

5 Conclusion

We introduced **VisualNeedle**, a 300-question “needle-in-a-haystack” benchmark whose construction closes three shortcuts that let MLLMs score high without genuinely engaging with the image: language priors, coarse global semantics, and near-invariance to the content of intermediate tool outputs. Across 9 mainstream models, no-tool accuracy stays below 20%; in the text-only diagnostic, accuracy stays below 10%. The human majority baseline is 63.00%, while the strongest evaluated model, Gemini 3.1 Pro, reaches only 56.01% on average even with visual tools. The matched crop-black setting further shows that real crops improve stronger models by 18.00–43.33 points over black crops on VisualNeedle. By contrast, all models in the V* diagnostic still reach 75.92%–87.43% under the crop-black setting, and several strong HR-Bench results remain at or above 82.00%, showing that prior benchmarks can be largely solved with limited dependence on intermediate visual content. VisualNeedle thus provides a diagnostic, shortcut-resistant testbed for active visual search and clearer evaluation targets for future visual agent systems.

References

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: A visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 23716–23736, 2022. URL <https://arxiv.org/abs/2204.14198>.

- Mohammad Asadi, Jack W. O’Sullivan, Fang Cao, Tahoura Nedaei, Kamyar Rajabalifardi, Fei-Fei Li, Ehsan Adeli, and Euan Ashley. MIRAGE: The illusion of visual understanding. *arXiv preprint arXiv:2603.21687*, 2026. doi: 10.48550/arXiv.2603.21687. URL <https://arxiv.org/abs/2603.21687>.
- ByteDance Seed Team. Official release of seed1.8: A generalized agentic model. <https://seed.bytedance.com/en/blog/official-release-of-seed1-8-a-generalized-agentic-model>, 2025. Official blog post, accessed 2026-04-03.
- ByteDance Seed Team. Seed 2.0 official launch. <https://seed.bytedance.com/en/blog/seed2-0-%25E6%25AD%25A3%25E5%25BC%258F%25E5%258F%2591%25E5%25B8%2583>, 2026. Official blog post, accessed 2026-04-03.
- Sicheng Feng, Song Wang, Shuyi Ouyang, Lingdong Kong, Zikai Song, Jianke Zhu, Huan Wang, and Xinchao Wang. ReasonMap: Towards fine-grained visual reasoning from transit maps. *arXiv preprint arXiv:2505.18675*, 2026. doi: 10.48550/arXiv.2505.18675. URL <https://arxiv.org/abs/2505.18675>.
- Chaoyou Fu, Yujie Dai, Yilun Luo, Shuhuai Ren, Runpei Zhang, Zihan Wang, Aojun Zhou, Mengdan Shen, Zhuotao Zhang, Can Xu, Zhengxiong Li, Meng Zhao, Chen Zhu, Yadong Liu, Shilong Wang, Jun Li, Yao Sun, Ran He, and Jifeng Li. MME: A comprehensive evaluation benchmark for multimodal large language models. *arXiv preprint arXiv:2306.13394*, 2023. doi: 10.48550/arXiv.2306.13394. URL <https://arxiv.org/abs/2306.13394>.
- Paul Gavrikov, Wei Lin, M. Jehanzeb Mirza, Soumya Jahagirdar, Muhammad Huzaifa, Sivan Doveh, Serena Yeung-Levy, James Glass, and Hilde Kuehne. Visualoverload: Probing visual understanding of VLMs in really dense scenes. *arXiv preprint arXiv:2509.25339*, 2025. doi: 10.48550/arXiv.2509.25339. URL <https://arxiv.org/abs/2509.25339>.
- Google. A new era of intelligence with gemini 3. <https://blog.google/products/gemini/gemini-3>, 2025. Official blog post, accessed 2026-04-03.
- Google. Gemini 3.1 pro: A smarter model for your most complex tasks. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro>, 2026. Official blog post, accessed 2026-04-03.
- Rynaa Grover, Jayant Sravan Tamarapalli, Sahiti Yerramilli, and Nilay Pande. HueManity: Probing fine-grained visual perception in MLLMs. *arXiv preprint arXiv:2506.03194*, 2026. doi: 10.48550/arXiv.2506.03194. URL <https://arxiv.org/abs/2506.03194>.
- Xingang Guo, Utkarsh Tyagi, Advait Gosai, Paula Vergara, Jayeon Park, Ernesto Gabriel Hernández Montoya, Chen Bo Calvin Zhang, Bin Hu, Yunzhong He, Bing Liu, and Rakshith Sharma Srinivasa. Beyond seeing: Evaluating multimodal LLMs on tool-enabled image perception, transformation, and reasoning. *arXiv preprint arXiv:2510.12712*, 2025. doi: 10.48550/arXiv.2510.12712. URL <https://arxiv.org/abs/2510.12712>.
- Jack Hong, Bing Li, Michael Yang, Chenxiao Zhao, Ziwei Zheng, Guohai Xu, Le Yang, Chao Shen, and Xing Yu. DeepEyesV2: Toward agentic multimodal model. *arXiv preprint arXiv:2511.05271*, 2025. doi: 10.48550/arXiv.2511.05271. URL <https://arxiv.org/abs/2511.05271>.
- Aditya Kanade and Tanuja Ganu. Do you see me: A multidimensional benchmark for evaluating visual perception in multimodal LLMs. *arXiv preprint arXiv:2506.02022*, 2025. doi: 10.48550/arXiv.2506.02022. URL <https://arxiv.org/abs/2506.02022>.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 19730–19742. PMLR, 2023. URL <https://arxiv.org/abs/2301.12597>.
- Ming Li, Jike Zhong, Shitian Zhao, Haoquan Zhang, Shaoheng Lin, Yuxiang Lai, Wei Chen, Konstantinos Psounis, and Kaipeng Zhang. TIR-Bench: A comprehensive benchmark for agentic thinking-with-images reasoning. *arXiv preprint arXiv:2511.01833*, 2025a. doi: 10.48550/arXiv.2511.01833. URL <https://arxiv.org/abs/2511.01833>.

- Siqi Li, Xinyu Cai, Jianbiao Mei, Nianchen Deng, Pinlong Cai, Licheng Wen, Yufan Shen, Xuemeng Yang, Botian Shi, and Yong Liu. UR-Bench: A benchmark for multi-hop reasoning over ultra-high-resolution images. *arXiv preprint arXiv:2601.08748*, 2025b. doi: 10.48550/arXiv.2601.08748. URL <https://arxiv.org/abs/2601.08748>.
- Fangyu Liu, Kevin Lin, Linjie Li, Jianfeng Wang, Yaser Yacoob, and Lijuan Wang. MMBench: Is your multi-modal model an all-around player? *arXiv preprint arXiv:2307.06281*, 2023a. doi: 10.48550/arXiv.2307.06281. URL <https://arxiv.org/abs/2307.06281>.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, volume 36, pages 34892–34916, 2023b. URL <https://arxiv.org/abs/2304.08485>.
- Zujing Liu, Junwen Pan, Qi She, Yuan Gao, and Gui-Song Xia. On the faithfulness of visual thinking: Measurement and enhancement. *arXiv preprint arXiv:2510.23482*, 2025. doi: 10.48550/arXiv.2510.23482. URL <https://arxiv.org/abs/2510.23482>.
- Yan Ma, Weiyu Zhang, Tianle Li, Linge Du, Xuyang Shen, and Pengfei Liu. What does vision tool-use reinforcement learning really learn? disentangling tool-induced and intrinsic effects for crop-and-zoom. *arXiv preprint arXiv:2602.01334*, 2026. doi: 10.48550/arXiv.2602.01334. URL <https://arxiv.org/abs/2602.01334>.
- Moonshot AI. Kimi k2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026. doi: 10.48550/arXiv.2602.02276. URL <https://arxiv.org/abs/2602.02276>.
- OpenAI. Introducing GPT-5.2. <https://openai.com/index/introducing-gpt-5-2>, 2025. Official release page, accessed 2026-04-03.
- OpenAI. Introducing GPT-5.4. <https://openai.com/index/introducing-gpt-5-4/>, 2026. Official release page, accessed 2026-04-03.
- Qwen Team. Qwen3.5-397b-a17b. <https://huggingface.co/Qwen/Qwen3.5-397B-A17B>, 2026a. Official model card for the hosted Qwen3.5-Plus, accessed 2026-04-03.
- Qwen Team. Qwen3-vl-235b-a22b-thinking. <https://huggingface.co/Qwen/Qwen3-VL-235B-A22B-Thinking>, 2026b. Official model card, accessed 2026-04-03.
- Karahan Sarıtaş, Peter Dayan, Tingke Shen, and Surabhi S. Nath. Complexity in complexity: Understanding visual complexity through structure, color, and surprise. *arXiv preprint arXiv:2501.15890*, 2025. doi: 10.48550/arXiv.2501.15890. URL <https://arxiv.org/abs/2501.15890>.
- Jinming Su, Dongxiang Wang, Yunzi Hao, Zehua Rao, Geng Chen, Kaijie Zhu, Bingning Dai, Zhaocheng Xu, Kai Xiong, Wenbo Ren, Jiayu Feng, Weiming Liu, Meng Yu, Zhou Yu, Hao Zhao, Wei Chen, Zhiheng Li, Junnan Dong, Bo Li, Yuchao Dai, Shuai Wang, Zhe Gan, Yu Qiao, Shuicheng Yan, and Biao Jiang. Thinking with images for multimodal reasoning: Foundations, methods, and future frontiers. *arXiv preprint arXiv:2506.23918*, 2025. doi: 10.48550/arXiv.2506.23918. URL <https://arxiv.org/abs/2506.23918>.
- Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. Eyes wide shut? exploring the visual shortcomings of multimodal LLMs. *arXiv preprint arXiv:2401.06209*, 2024. doi: 10.48550/arXiv.2401.06209. URL <https://arxiv.org/abs/2401.06209>.
- Wenbin Wang, Liang Ding, Minyan Zeng, Xiabin Zhou, Li Shen, Yong Luo, and Dacheng Tao. Divide, conquer and combine: A training-free framework for high-resolution image perception in multimodal large language models. *arXiv preprint arXiv:2408.15556*, 2024. doi: 10.48550/arXiv.2408.15556. URL <https://arxiv.org/abs/2408.15556>.
- Penghao Wu and Saining Xie. V*: Guided visual search as a core mechanism in multimodal LLMs. *arXiv preprint arXiv:2312.14135*, 2024. doi: 10.48550/arXiv.2312.14135. URL <https://arxiv.org/abs/2312.14135>.
- Hong-Tao Yu, Yuxin Peng, Serge Belongie, and Xiu-Shen Wei. Benchmarking large vision-language models on fine-grained image tasks: A comprehensive evaluation. *arXiv preprint arXiv:2504.14988*, 2026. doi: 10.48550/arXiv.2504.14988. URL <https://arxiv.org/abs/2504.14988>.

- Xiang Yue, Yulan Ni, Kai Zhang, Tianyu Zheng, Ruichen Li, Ge Liu, Weiming Huang, Huan Sun, and Yu Su. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI. *arXiv preprint arXiv:2311.16502*, 2023. doi: 10.48550/arXiv.2311.16502. URL <https://arxiv.org/abs/2311.16502>.
- Yi-Fan Zhang, Xingyu Lu, Shukang Yin, Chaoyou Fu, Wei Chen, Xiao Hu, Bin Wen, Kaiyu Jiang, Changyi Liu, Tianke Zhang, Haonan Fan, Kaibing Chen, Jiankang Chen, Haojie Ding, Kaiyu Tang, Zhang Zhang, Liang Wang, Fan Yang, Tingting Gao, and Guorui Zhou. Thyme: Think beyond images. *arXiv preprint arXiv:2508.11630*, 2025. doi: 10.48550/arXiv.2508.11630. URL <https://arxiv.org/abs/2508.11630>.
- Yipeng Zhang, Xiaoqing Wang, Jiayi Jin, Yidong Li, Zhongwei Luo, Yixin Chen, Meng Li, Qing Wang, and Can Xu. MME-RealWorld: Could your multimodal LLM challenge high-resolution real-world scenarios that are difficult for humans? *arXiv preprint arXiv:2408.13257*, 2024. doi: 10.48550/arXiv.2408.13257. URL <https://arxiv.org/abs/2408.13257>.
- Ziwei Zheng, Michael Yang, Jack Hong, Chenxiao Zhao, Guohai Xu, Le Yang, Chao Shen, and Xing Yu. Deepeyes: Incentivizing “thinking with images” via reinforcement learning. *arXiv preprint arXiv:2505.14362*, 2025. doi: 10.48550/arXiv.2505.14362. URL <https://arxiv.org/abs/2505.14362>.
- Xuanyu Zhu, Yuhao Dong, Rundong Wang, Yang Shi, Zhipeng Wu, Yinlun Peng, YiFan Zhang, Yihang Lou, Yuanxing Zhang, Ziwei Liu, Yan Bai, and Yuan Zhou. Vtc-bench: Evaluating agentic multimodal models via compositional visual tool chaining. *arXiv preprint arXiv:2603.15030*, 2026. doi: 10.48550/arXiv.2603.15030. URL <https://arxiv.org/abs/2603.15030>.

A Limitations

VisualNeedle is designed as a diagnostic benchmark for active visual search in information-dense scenes, but it has several limitations. First, the benchmark contains 300 carefully curated questions, which enables detailed annotation and trajectory analysis but cannot exhaustively cover all real-world visual search scenarios, languages, cultural contexts, or image sources. Second, several evaluated systems are closed-source API models, so their results may be affected by model-version updates, service-side changes, and sampling stochasticity. Third, because VisualNeedle uses open-ended questions, normalized exact matching alone is insufficient for answer evaluation; therefore, we use an independent visual judge after exact matching, which improves coverage but may still introduce small errors for ambiguous visual evidence, equivalent answer expressions, or borderline cases.

B Implementation Details

In implementation, we use a unified agent evaluation framework for all experiments. In the tool-enabled setting, a model receives the full image as its initial input and can call a visual crop tool over multiple turns to progressively zoom into local regions. In the no-tool setting, tool-calling ability is removed, and the model must answer directly from the full image. To avoid state contamination across samples, we instantiate a fresh agent for each sample and assign it an independent tool working directory. Input images are converted to local files before evaluation and compressed to no more than 36M pixels when necessary. During evaluation, we record the parameters and outputs of each tool call as well as derived images, preserving complete search trajectories. Answer judgment follows a two-stage strategy: we first perform normalized exact matching; if rule-based matching fails, we then call an independent visual judge, which determines whether the model output is semantically correct given the original image, question, ground-truth answer, and model response.

Sample format. VisualNeedle is stored as a JSONL dataset with one question per line. Each sample contains a unique annotation identifier `id`, an image identifier `tag`, a local image path `image`, a public image source `image_url`, the question `question`, the ground-truth answer `answer`, category labels `class1`, `class2`, and `class3`, a formatting flag `is_markdown`, and a ground-truth target box `bbox`. The released evaluation split used in this paper does not include a separate alternative-answer field; equivalent answer variants are handled by the answer normalizer and the independent visual judge described below. The target box is stored in the original image coordinate frame, not in the resized model-input frame. Its format is $[x1, y1, x2, y2]$, where $(x1, y1)$ is the top-left corner and $(x2, y2)$ is the bottom-right corner. We use the standard half-open image-box convention for area, overlap, and cropping computations: valid boxes satisfy $0 \leq x1 < x2 \leq W$ and $0 \leq y1 < y2 \leq H$, so the right and bottom boundaries are treated as exclusive.

Evaluation settings. All models are evaluated in a zero-shot setting under four protocols. In *text-only*, the model receives only the question, with both image fields removed. In *w/o tools*, the model receives the full image but the tool interface is disabled, so it must answer from the initial global view. In *w/ tools*, the model receives the full image and may call the Crop tool over multiple turns; each Crop call specifies an `img_idx`, a `bbox_2d`, and a free-form `reason`, and returns the selected high-resolution local crop as the next visual observation. In the *crop-black* setting, the interaction and tool schema are identical to *w/ tools*, but every returned crop image is replaced in place by an all-black image with exactly the same pixel size as the true crop. Thus this intervention preserves the tool-call trajectory and output dimensions while removing the visual evidence inside the crop.

Termination and scoring. All models are connected to the same evaluation framework through a unified API adaptation layer, and we use the default Qwen-Agent maximum number of LLM interaction turns for a single run, namely 20 turns. A run terminates when the model emits a final answer without a pending tool call or when the interaction budget is exhausted. If the model does not produce a parsable answer, exceeds the per-sample timeout, repeatedly produces malformed or illegal tool calls that cannot be recovered by the evaluation harness, or terminates without an answer, the sample is counted as incorrect. For answer judgment, we first apply normalized exact matching. If exact matching fails, we use an independent visual judge, Gemini 3.1 Pro, which receives the original image, the question, the ground-truth answer, the parsed answer, and the full model response. The

judge returns a binary correctness decision; only a positive decision changes an exact-match failure into a correct prediction, and invalid or negative judge outputs are counted as incorrect.

The specific generation hyperparameters are shown in Table 4.

Table 4: Generation hyperparameters used in evaluation.

Model(s)	Hyperparameters
GPT-5.2, GPT-5.4	<code>temperature = 1.0, reasoning_effort = xhigh.</code>
Gemini 3.1 Pro, Gemini 3 Flash	<code>temperature = 1.0, thinking_level = HIGH, max_tokens = 65,536.</code>
Doubao-Seed-1.8	<code>temperature = 1.0, thinking enabled, high reasoning mode, max_tokens = 32,000.</code>
Doubao-Seed-2.0-Pro	<code>temperature = 1.0, thinking enabled, high reasoning mode, max_tokens = 128,000.</code>
Kimi K2.5	<code>temperature = 1.0, thinking enabled, max_tokens = 128,000.</code>
Qwen3.5-Plus	<code>temperature = 1.0, thinking enabled, high-resolution image input enabled.</code>
Qwen3-VL-235B-A22B-Thinking	<code>temperature = 0.7, top_p = 0.8, top_k = 20, repetition_penalty = 1.0, presence_penalty = 1.5, high-resolution image input enabled.</code>

C Human Annotation Protocol

To ensure a rigorous and unbiased assessment, we recruited an independent evaluation team consisting of 12 professional annotators. Each team member possesses extensive experience in multimodal data labeling and reasoning. Importantly, these annotators were held blind to all preceding stages of the pipeline, including image collection, question authoring, bounding-box annotation, and quality filtering. This separates the human-baseline answering stage from the data-construction and verification stage described in Section 3.2. Each evaluation item was answered independently by 3 different annotators, yielding 900 human annotations in total. Annotators saw one image–question pair at a time and submitted a single best answer in the same answer format expected from the models. They were not shown model-generated hints, answers from other annotators, ground-truth answers, target boxes, or other construction-time metadata.

All annotators used the same web-based annotation platform used for human evaluation, with the same user interface, image viewer, and shortcut bindings. The image viewer allowed unrestricted zooming and panning over the original-resolution image, so annotators could inspect arbitrary local regions before answering. Human annotators did not call the model-side Crop API; their interface provided the human-side analogue of free local visual search through zoom and pan. No annotator used a custom client or external image editor. For fairness, annotators were instructed to spend at most 5 minutes per item; if they could not locate the answer with reasonable confidence after this budget, they were asked to submit their best guess or mark the item as suspended. In the final 300-item set, all assigned items received actual answers, giving a 100.00% completion rate and a 0.00% suspension rate. Platform logs record the assigned annotator, start/end timestamps, elapsed time, submitted answer, and suspension flag for each item.

D Judge Audit and Case Analysis

All evaluation samples come with reference answers (GT), and the data construction process aims to make each answer as unique and unambiguous as possible. For answer scoring, we first apply normalized exact matching. If exact matching fails, Gemini 3.1 Pro performs a VLM-based semantic check using the image, question, model prediction, and reference answer. In other words, the judge combines semantic matching with reference-answer analysis rather than relying on string equality alone.

To sanity-check the judge, we asked three human quality-control reviewers to audit a sample of judge decisions. The average agreement between Gemini 3.1 Pro and the human audit is 93.67%.

Representative cases are shown below.

Table 5: Human annotation protocol for the human-baseline evaluation on the final 300-item split.

Aspect	Protocol
Annotator pool	12 external annotators; no overlap with the data-construction team.
Annotations per item	3 independent annotators per item, yielding 900 annotations in total.
Task input	One high-resolution image and its natural-language question.
Task output	One answer string per item; no rationale required.
Hidden information	No model hints, other annotators’ answers, ground-truth answers, target boxes, difficulty tags, or filtering metadata.
Zoom / crop access	Unrestricted zoom and pan in the platform image viewer; no model-side Crop API, external image editor, or custom client.
Shared human interface	All annotators used the same web-based human-evaluation platform, UI, viewer, and shortcuts.
Time budget	At most 5 minutes per item; submit best guess or suspend if unresolved.
Measured time	Mean per-annotator time ranged from 2 min 37 s to 3 min 35 s; overall mean was approximately 3 min 17 s per item.
Completion / suspension	100.00% completion rate and 0.00% suspension rate on the final 300-item set.
Reported metrics	Majority Vote@3: 189/300 = 63.00%; Pass@3: 224/300 = 74.67%.

Good case: hard but judged correctly

Question. What are the characters on the sign to the right of “裘记单车修理铺”?

Prediction. 诚兴麻雀公司

Gold. 誠興麻雀公司

Good case: hard translation equivalent

Question. What is the name of the sixth book on the shelf below “朝花夕拾”?

Prediction. 百年孤独

Gold. One Hundred Years of Solitude

Bad case: false negative

Question. In the image, what characters are written on the sycee held by the lucky cat graffiti to the left of the man wearing a hat?

Prediction. 開運招財

Gold. 開運

Bad case: false positive

Question. What is the second item to the left of the monkey in the picture?

Prediction. A Hello Kitty doll

Gold. Cat

E Repeated-run Stability Analysis

To assess run-to-run stability, we repeat the main no-tool and tool-enabled evaluations three times under the same experimental setting. For each metric x , Table 6 and Table 7 report the mean, sample standard deviation, and 95% confidence interval half-width:

$$\bar{x} = \frac{1}{3} \sum_{i=1}^3 x_i, \quad s = \sqrt{\frac{1}{2} \sum_{i=1}^3 (x_i - \bar{x})^2}, \quad \text{CI}_{95} = t_{0.975,2} \frac{s}{\sqrt{3}},$$

where $t_{0.975,2} = 4.303$. These intervals capture stochastic variation across repeated evaluations on the fixed 300-question split, rather than uncertainty from resampling the benchmark itself. Because only three repetitions are available, the confidence intervals should be interpreted as a conservative stability diagnostic rather than as a definitive population-level estimate.

Table 6: Run-to-run variance for accuracy metrics. Each cell in the mean $\pm$ std. columns reports the mean and sample standard deviation across three repeated runs. CI_{95} reports the 95% confidence interval half-width.

Model	w/ Tools	w/ Tools	w/o Tools	w/o Tools	Δ	Δ
	Mean $\pm$ Std.	CI_{95}	Mean $\pm$ Std.	CI_{95}	Mean $\pm$ Std.	CI_{95}
Gemini 3.1 Pro	56.01 $\pm$ 0.69	1.70	16.34 $\pm$ 0.87	2.16	39.67 $\pm$ 1.53	3.79
Gemini 3 Flash	48.00 $\pm$ 1.48	3.68	15.12 $\pm$ 1.70	4.24	32.88 $\pm$ 2.38	5.92
Doubao-Seed-2.0-Pro	36.80 $\pm$ 1.82	4.53	13.11 $\pm$ 1.17	2.90	23.69 $\pm$ 2.00	4.97
Doubao-Seed-1.8	31.78 $\pm$ 1.38	3.42	17.89 $\pm$ 1.02	2.53	13.89 $\pm$ 1.69	4.19
GPT-5.4	31.23 $\pm$ 1.08	2.68	14.32 $\pm$ 0.67	1.65	16.88 $\pm$ 1.70	4.24
GPT-5.2	28.31 $\pm$ 1.71	4.26	10.56 $\pm$ 1.50	3.73	17.75 $\pm$ 2.35	5.85
Qwen3.5-Plus	22.11 $\pm$ 0.36	0.88	15.00 $\pm$ 1.76	4.37	7.11 $\pm$ 1.54	3.83
Kimi K2.5	19.32 $\pm$ 3.17	7.87	10.78 $\pm$ 1.49	3.69	8.55 $\pm$ 2.02	5.01
Qwen3-VL-235B-A22B-Thinking	11.44 $\pm$ 0.81	2.00	8.34 $\pm$ 0.65	1.61	3.10 $\pm$ 0.17	0.43

Table 7: Tool-call stability across repeated runs. Avg. Tool Calls is the per-question average number of Crop calls, and Total Tool Calls is the total number of Crop calls over all 300 questions.

Model	Avg. Tool Calls	Avg. Tool Calls	Total Tool Calls	Total Tool Calls
	Mean $\pm$ Std.	CI_{95}	Mean $\pm$ Std.	CI_{95}
Gemini 3.1 Pro	3.72 $\pm$ 0.08	0.19	1117.33 $\pm$ 22.12	54.95
Gemini 3 Flash	3.90 $\pm$ 0.10	0.25	1174.00 $\pm$ 31.76	78.91
Doubao-Seed-2.0-Pro	6.42 $\pm$ 0.14	0.36	1923.00 $\pm$ 41.62	103.38
Doubao-Seed-1.8	4.77 $\pm$ 0.11	0.27	1435.67 $\pm$ 28.22	70.10
GPT-5.4	11.14 $\pm$ 0.35	0.86	3339.33 $\pm$ 103.65	257.47
GPT-5.2	9.30 $\pm$ 0.26	0.66	2795.67 $\pm$ 73.71	183.11
Qwen3.5-Plus	3.51 $\pm$ 0.73	1.82	1056.67 $\pm$ 214.26	532.26
Kimi K2.5	6.59 $\pm$ 0.34	0.86	1974.33 $\pm$ 111.13	276.07
Qwen3-VL-235B-A22B-Thinking	1.59 $\pm$ 0.28	0.69	473.67 $\pm$ 78.82	195.80

The repeated runs support the main qualitative conclusions. Gemini 3.1 Pro and Gemini 3 Flash consistently form the leading group among the 9 evaluated models, while Qwen3-VL-235B-A22B-Thinking shows only a small positive gain with very low run-to-run variation. In contrast, GPT-5.4 and GPT-5.2 have consistently high tool-call counts, confirming that more interaction alone is insufficient for effective active visual search.

F Interactive Search Process

This section provides a more formal description of the interactive search process in VisualNeedle. Specifically, at step t , the model chooses an action $a_t = (o_t, p_t)$, where o_t denotes the operation selected at the current step and p_t denotes the parameters associated with that operation. The action space is $\mathcal{A} = \{\mathbf{Crop}, \mathbf{Answer}\}$. When the model chooses **Crop**, the parameter p_t is the bounding box $bbox_t$; when it chooses **Answer**, the parameter p_t is the final answer y .

Algorithm 1 Interactive search process in VisualNeedle.

Require: high-resolution image I , question Q , action space $\mathcal{A} = \{\mathbf{Crop}, \mathbf{Answer}\}$

```

1:  $V_0 \leftarrow I$ ,  $t \leftarrow 0$ 
2: while True do
3:    $(o_t, p_t) \leftarrow \pi(V_t, Q)$ 
4:   if  $o_t = \mathbf{Crop}$  then
5:      $bbox_t \leftarrow p_t$ 
6:      $V_{t+1} \leftarrow \text{RESIZEIFNEEDED}(I[bbox_t])$ 
7:      $t \leftarrow t + 1$ 
8:   else if  $o_t = \mathbf{Answer}$  then
9:      $y \leftarrow p_t$ 
10:    return  $y$ 
11:   end if
12: end while

```

G Category-wise Results

Table 8 reveals task-difficulty differences in VisualNeedle from a category perspective. In terms of the upper bound of the best model, spatial relation recognition and OCR are relatively easier, with best accuracies of 72.41% and 66.20%, respectively. By contrast, the best accuracies for entity recognition, color recognition, and occluded object recognition are below 50%, indicating that current models remain clearly limited when tasks require more fine-grained target confirmation or more complex local reasoning. From the average performance of the 9 models, occluded object recognition is the hardest category, with an average tool-enabled accuracy of only 23.53%, the lowest among all categories. Its average no-tool accuracy is also only 8.17%, suggesting that even with the Crop tool, models still struggle to reliably integrate local clues for occluded targets, making occlusion reasoning a key weakness of current visual agents. At the same time, OCR has the highest average tool gain, reaching 26.92 percentage points, showing that tools are most effective at alleviating the problem of unclear local details. The average gain for color recognition is only 10.81 percentage points, indicating that in some tasks the bottleneck is no longer only search and zooming, but also subtle attribute discrimination itself.

Figure 5 further illustrates model capability distributions across the five categories. Overall, Gemini 3.1 Pro leads across all five categories, with particularly clear advantages in OCR and spatial relation recognition. By contrast, GPT-5.4, GPT-5.2, Doubao-Seed-1.8, Qwen3.5-Plus, and other models show larger performance variation across categories: they can maintain certain levels on OCR and spatial relation recognition, but their accuracy drops more noticeably once tasks shift toward entity recognition and occluded object recognition. Meanwhile, Kimi K2.5 and Qwen3-VL remain lower overall and show more unstable performance across categories, indicating more systematic weaknesses in fine-grained localization, local-evidence integration, and target confirmation.

Table 8: Category-wise VisualNeedle results. Best Acc. denotes the highest accuracy among the 9 models in the tool-enabled setting. Avg. W/O, Avg. W/, and Gain denote the average no-tool accuracy, average tool-enabled accuracy, and average tool gain across the 9 models in each category. Because of annotation-boundary differences, one borderline sample is included in the Entity Recognition category.

Category	#Q	Best Model	Best Acc. (%)	Avg. W/O (%)	Avg. W/ (%)	Gain (%)
OCR	71	Gemini 3.1 Pro	66.20	11.42	38.34	26.92
Spatial Relation	58	Gemini 3.1 Pro	72.41	13.79	34.29	20.50
Entity Recognition	64	Gemini 3.1 Pro	48.44	13.89	27.95	14.06
Color Recognition	73	Gemini 3.1 Pro	43.84	17.66	28.46	10.81
Occluded Object Recognition	34	Gemini 3.1 Pro	41.18	8.17	23.53	15.36

H Comparison Across Tool Configurations

To analyze whether introducing more tools can bring additional performance gains in VisualNeedle, we further design a multi-tool comparison experiment. Specifically, we compare the minimal Crop-only tool configuration with a multi-tool configuration that additionally includes enhancement and annotation operations from OpenCV. To clarify the concrete composition of this multi-tool setting, Table 9 lists the OpenCV tools used in this comparison experiment and their definitions. Table 10 shows that for Gemini 3.1 Pro, Gemini 3 Flash, and Doubao-Seed-2.0-Pro, the crop-only setting reaches 55.33%, 47.00%, and 38.00%, respectively, all higher than the 53.00%, 42.67%, and 31.00% achieved in the multi-tool setting. This indicates that in VisualNeedle, the main performance gain comes from the local high-resolution observation provided by Crop rather than from adding more image-processing operations. One possible reason is that the core bottleneck of the benchmark lies in tiny regions related to the answer. After localization, additional enhancement, annotation, or transformation operations may not provide new key information, and may instead enlarge the action space and increase the decision burden. Therefore, the minimal tool framework centered on Crop is already sufficient to support effective search, while the more complex tool combination brings no additional gain in this comparison experiment.

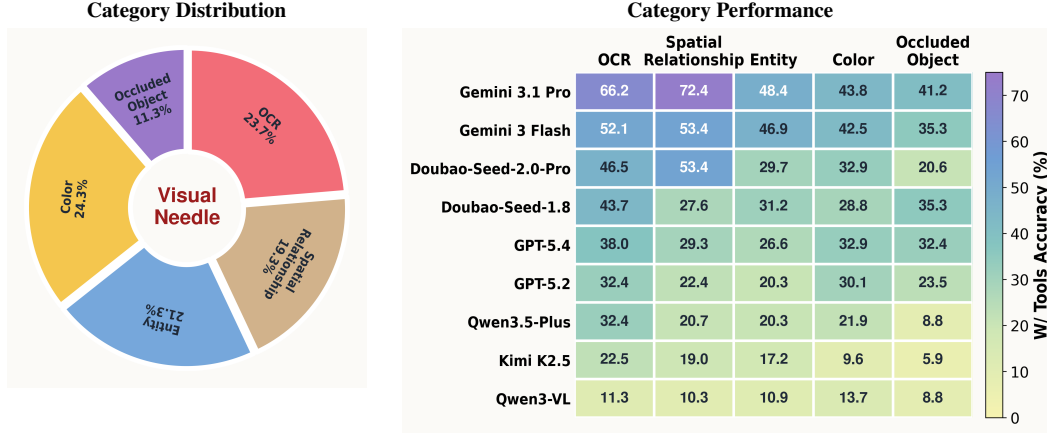


Figure 5: Category overview of VisualNeedle. The left panel shows the data distribution across the five categories, and the right panel shows a heatmap of tool-enabled accuracy for each model across the five categories. Rows correspond to models and columns correspond to categories, with colors and cell values jointly reflecting model performance across different fine-grained ability dimensions.

I Case Study of Search Trajectories

Figures 6, 7, and 8 present three representative search trajectories, corresponding to the **OCR**, **Entity**, and **Occluded Object** abilities. The OCR example requires models to locate an extremely small text region in an information-dense street scene and recognize the local characters after hitting the target. The Entity example requires precise localization and confirmation among multiple visually similar candidate objects. The Occluded Object example further requires models to identify a target under partial occlusion and incomplete evidence. Together, these cases show that the difficulty of VisualNeedle lies not only in seeing the target, but also in quickly narrowing the search region in complex scenes and stably converting local visual evidence into the final answer.

In terms of model behavior, Gemini 3.1 Pro shows faster early localization and more stable convergence in all three examples. For instance, in the OCR example, it hits the target and answers correctly at step 3, whereas GPT-5.2 does not hit the target until step 17 and still answers incorrectly. In the Entity example, Gemini 3.1 Pro hits the target and correctly identifies it at step 2, while GPT-5.2 hits it only at step 14 and still fails to confirm it correctly. In the Occluded Object example, Gemini 3.1 Pro hits the target and answers correctly at step 3, whereas GPT-5.2, Qwen3.5-Plus, and Doubao-Seed-2.0-Pro never truly hit the target region, indicating that only a few models can perform effective search under occlusion. Overall, GPT-5.2 often needs a longer probing process and may still fail to answer correctly even after hitting the target. Qwen3.5-Plus is more likely to stop early before entering an effective region. Doubao-Seed-2.0-Pro lies between the two, sometimes gradually approaching the target but remaining unstable in evidence integration and final confirmation.

Table 9: OpenCV tools and definitions used in the multi-tool comparison experiment. We divide these tools into four groups: geometry, image enhancement, feature and structure analysis, and drawing and measurement.

Tool Name	Description
Geometry	
Crop	Extracts a sub-region of the image specified by a bounding box, producing a high-resolution local view for fine-grained inspection.
Resize	Resizes an image to a target resolution.
Rotate	Rotates the image by a specified angle.
Translate	Shifts the image along the horizontal or vertical direction.
Flip	Mirrors the image horizontally, vertically, or both.
Pyramid	Performs pyramid upsampling or downsampling.
Enhancement	
Convert Color (Gray / HSV / LAB)	Converts the image to another color space for downstream processing.
In-Range Color	Filters pixels within a specified color range.
Blur	Applies smoothing filters to suppress high-frequency noise.
Denoise	Removes image noise while preserving salient structures.
Threshold	Produces binary or adaptive thresholded outputs.
Morphology	Applies erosion, dilation, opening, and closing operations.
Histogram	Computes histogram-based image statistics and contrast enhancement.
ConvertScaleAbs	Adjusts brightness and contrast through absolute scaling.
Inpaint	Restores missing or corrupted regions in the image.
Feature and Structure Analysis	
Canny	Detects edges using the Canny operator.
Gradients	Computes first- or second-order image gradients.
Contours	Extracts contour structures from binary images.
Contour Area	Computes the area of a contour.
Arc Length	Computes the perimeter or curve length of a contour.
Approx Poly	Approximates a contour with a polygon of fewer vertices.
Watershed	Segments touching regions with watershed partitioning.
GrabCut	Extracts foreground objects via graph-cut-based segmentation.
Flood Fill	Expands a connected region from a seed point.
Connected Components	Computes connected components and their statistics.
Features	Detects local keypoints and descriptors.
Hough Lines	Detects straight lines with the Hough transform.
Hough Circles	Detects circular patterns with the Hough transform.
Template Match	Locates image regions matching a template.
DFT	Converts the image to the frequency domain.
Drawing and Annotation	
Draw Line	Draws a straight line on the image.
Draw Circle	Draws a circle on the image.

Table 10: Results of the multi-tool comparison experiment on VisualNeedle.

Model	w/ tool (crop-only)	w/ tool (multi-tools)
Gemini 3.1 Pro [Google, 2026]	55.33%	53.00%
Gemini 3 Flash [Google, 2025]	47.00%	42.67%
Doubao-Seed-2.0-Pro [ByteDance Seed Team, 2026]	38.00%	31.00%

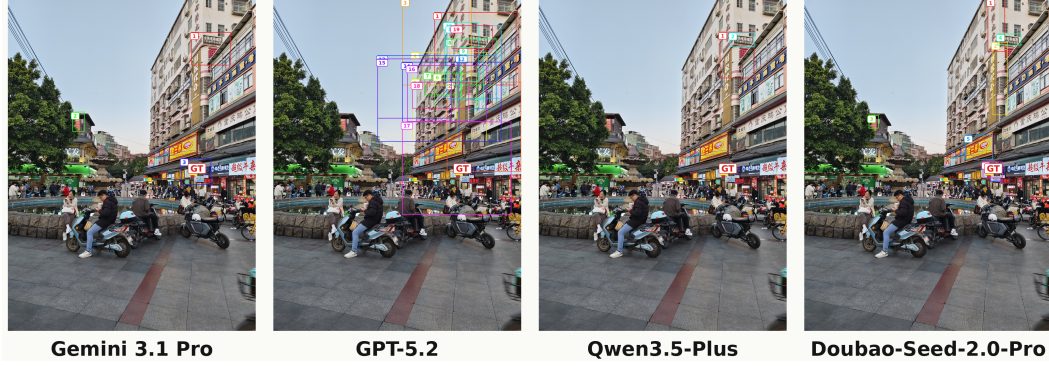


Figure 6: Comparison of four-model search trajectories on an OCR example. This example requires models to locate tiny text on a red sign in a complex street scene and recognize the characters. Gold boxes denote ground-truth target regions, colored boxes denote zoom boxes at each step, and numbers denote step indices.

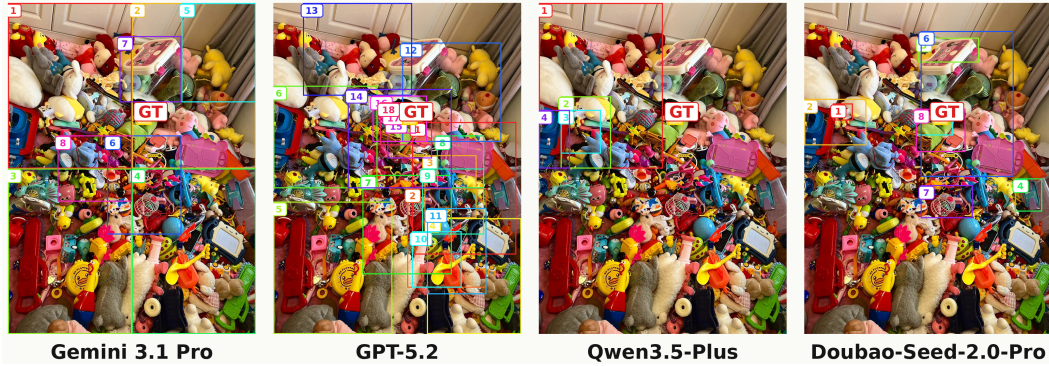


Figure 7: Comparison of four-model search trajectories on an Entity example. This example requires models to locate the blue target closest to a panda-patterned card among multiple similar small objects and complete entity confirmation. Gold boxes denote ground-truth target regions, colored boxes denote zoom boxes at each step, and numbers denote step indices.



Figure 8: Comparison of four-model search trajectories on an Occluded Object example. This example requires models to locate a posted item behind a shopkeeper’s wooden table under occlusion and background clutter, and then determine its function. Gold boxes denote ground-truth target regions, colored boxes denote zoom boxes at each step, and numbers denote step indices.

J Prompts

This section summarizes the prompt templates used in our evaluation scripts. The Crop setting does not use a separate natural-language prompt beyond the tool-enabled VQA prompt. Instead, Crop is exposed through the model-facing tool schema of `image_zoom_in_tool_reason`; the model is instructed by the main prompt to call the available zoom/crop tool when local details are unclear. We show the relevant templates below and omit only runtime values such as image paths, questions, model responses, and credentials.

Main experiment system prompt: tool-enabled VQA

System message.

You are a visual question answering assistant specializing in detailed image analysis. You have access to tools to help you inspect images more reliably. These tools may include (depending on the provided tool list): zoom/crop, flip/rotate, sharpening/contrast enhancement, and Python code execution (`code_interpreter`). Tools operate on the images in this conversation, including the original input image and any derived images returned by tools in earlier steps.

Structured Thinking Process. Follow this iterative process for each question:

1. First, look closely: describe the image in detail, identify what information you need to answer the question.
2. Identify regions of interest: if the answer depends on small/unclear details (text, numbers, fine print, charts, tables, small objects), identify the approximate region to zoom in.
3. Use an appropriate image tool: call the most suitable tool from the provided tool list. For tiny/unclear details, use a zoom/crop tool. For mirrored/upside-down content, use a flip/rotate tool. For blurry text, use sharpening/contrast tools. If existing tools are not sufficient, use `code_interpreter` to write Python for custom image processing and related logic; in the `code` argument, provide only Python code, and print the specific facts you need rather than relying on generated plots/images alone. Follow the tool schema exactly. If a tool supports selecting an image (e.g., via `img_idx`), use it to choose which image to operate on (0 usually refers to the original input image). If a tool requires a region (e.g., `bbox_2d`), use the coordinate format specified in the tool schema.
4. Analyze the tool result: carefully read the processed/zoomed view and extract relevant information.
5. Repeat if needed: you may apply tools multiple times. When operating on a derived image, ensure you select the correct image index (if applicable) and use coordinates relative to that image.

Tool-use Policy. Use tools proactively whenever you are not 100% confident about text, numbers, or small details. Prefer the least invasive tool that resolves the uncertainty (e.g., zoom before heavy enhancement). Do NOT guess; if you cannot read something clearly, use tools to verify. Always provide all required parameters for each tool call as specified in the tool schema.

Answer Policy. After completing your analysis (including any tool use), output ONLY the final answer text on the last line. Do NOT add any explanation, prefixes (e.g., “答案: ”), or quotes around the answer.

Main experiment user prompt: tool-enabled VQA

User message template.

Question: {question}

Instructions:

1. First, examine the full image and identify what information you need.
2. If the answer depends on text, numbers, small objects, or any detail that is not clearly visible, use the available image tools from the provided tool list to inspect/verify the relevant details. For tiny/unclear details, prefer zoom/crop tools. For mirrored/upside-down content, use flip/rotate tools. For blurry text, use sharpening/contrast tools. If multiple images are present in the conversation (e.g., original plus derived images), select the correct one if the tool supports it, often via `img_idx`.
3. After verifying the details, provide your answer.

Do NOT guess if you are uncertain; use the available image tools to verify. Output ONLY the final answer text on the last line.

Crop tool interface

Tool name. `image_zoom_in_tool_reason`

Model-facing description.

Zoom in on a specific region of an image. REQUIRED params: `reason` (string), `img_idx` (number; 0 refers to the first image visible in this conversation, subsequent indices refer to images returned by earlier tool calls in chronological order), `bbox_2d` (array [`x1`, `y1`, `x2`, `y2`] in 0–1000 coords). All three parameters must be provided.

Required fields.

reason: reasoning for this zoom. Include what you learned from previous zooms (if any), why this region needs inspection, and how this differs from previous zooms to avoid duplication.

img_idx: index of the image to reference. 0 is the first image visible to the model in the conversation; larger indices are images returned by previous tool calls in chronological order.

bbox_2d: the bounding box of the region to zoom in, as [`x1`, `y1`, `x2`, `y2`] in relative coordinates normalized to [0, 1000] (not pixels), where (`x1`, `y1`) is the top-left corner and (`x2`, `y2`) is the bottom-right corner.

Optional field. label: a descriptive label for the target region, such as `store_sign` or `person_with_purple_bag`.

Text-only diagnostic prompt

System message.

You are a visual question answering assistant. Answer the user's question based only on what you can observe in the image. Output ONLY the final answer text on the last line (no explanation).

User message template.

Question: {question}

You do NOT have access to any image. Based only on the question text and your prior knowledge, give your single best guess for the answer.

Output ONLY the final answer text on the last line.

Retry suffix if the first response is unparsable.

IMPORTANT: Output ONLY the final answer text on the last line. Do NOT add any explanation, prefixes (e.g., “答案: ”), or quotes.

Judge system prompt

System message.

Given a question and its golden label, judge whether the model's output correctly answers the question with a response that matches the golden label.

Ignore these differences (format tolerant). Markdown formatting: ***code***. Quotes, brackets, and parentheses: “答案” equals 答案; (工商舖) equals 工商舖. Extra explanation or preamble before/after the answer; whitespace and punctuation differences; case differences such as “hello” = “Hello”; simplified/traditional Chinese such as 工商舖 = 工商舖; units or measure words such as “3个” = “3” and “两台” = “2”; digit formatting such as “9736-8443” = “97368443”.

These differences mean NO (content strict). Any numeric difference, such as “3” ≠ “5”, “五楼” ≠ “二楼”, or “12” ≠ “13”. Different entities, such as 耳机 ≠ 手机, 北京 ≠ 上海, or Toyota ≠ Honda. Different color, such as 黑色 ≠ 白色. Different code/ID, such as “E-461236” ≠ “E-461237”.

Output. YES or NO.

Judge user prompt

Image-based judge user template.

The judge receives the original image together with the following text:

Question asked about the image: {question}

Ground-truth label answer: {gold_answer}

Model's parsed answer: {parsed_answer} or <NONE>

Model's full response: {model_response, truncated to 8000 chars}

Based on the image, question, and the answers above, is the model's answer correct? Output EXACTLY one word: YES or NO.